

SOUND SHARE

Documentación Técnica Completa

Plataforma de gestión musical sobre arquitectura de microservicios

Documentación Interna
Yeraldin Posada Gómez

Administración de sistemas informáticos
Universidad Nacional de Colombia
Sede Manizales
2026

Tabla de Contenidos

Tabla de Contenidos

1. Descripción General

1.1 Principios de diseño

1.2 Stack tecnológico por servicio

2. Arquitectura del Sistema

2.1 Diagrama lógico

2.2 Comunicación entre microservicios

2.3 Diagrama de Arquitectura

3. Despliegue Detallado

3.1 API Gateway (Laravel + MySQL)

3.2 Songs Service (Django + MySQL)

3.3 Downloads Service (Laravel + PostgreSQL)

3.4 Playlists Service (Flask + Firebase)

3.5 Interactions Service (Express + MongoDB)

3.6 Lyrics Service (Flask + MySQL)

4. Variables de Entorno

4.1 API Gateway

4.2 Microservicios (Songs, Playlists, Downloads, Lyrics)

4.3 Interactions Service

5. Referencia de Endpoints

5.1 Autenticación

Ejemplo: POST /api/login

5.2 Songs

5.3 Playlists

5.4 Interactions

5.5 Downloads

5.6 Lyrics

6. Pruebas de Rendimiento

6.1 Instalación y ejecución de Locust

6.2 Estructura de los scripts Locust

6.3 Resultados — Downloads Service

6.4 Resultados — Interactions Service

6.5 Resultados — Lyrics Service

6.6 Resultados — Playlists Service

6.7 Resultados — Songs Service

Prueba de carga (comportamiento normal)

7. Análisis y Conclusiones

[7.1 Punto de saturación del sistema](#)

[7.2 Comparativa de rendimiento entre servicios](#)

[8. Pruebas Unitarias](#)

[8.1 Contexto general](#)

[8.2 Playlists Service — Flask + Firebase \(pytest\)](#)

[8.3 Gateway — Laravel + Sanctum \(PHPUnit\)](#)

[8.4 Interactions Service — Express + MongoDB \(Jest + Supertest\)](#)

[9.Despliegue con Docker](#)

[9.1 Contexto general](#)

[9.2 Infraestructura de bases de datos](#)

[9.3 Docker files por microservicio](#)

[9.4 Variables de entorno por servicio](#)

[9.5 Orden de arranque y dependencias](#)

[9.6 Problemas encontrados y soluciones](#)

[9.7 Despliegue](#)

[10. Conclusiones](#)

1. Descripción General

Sound Share es una plataforma de gestión musical construida sobre una arquitectura de microservicios desacoplada. Cada servicio es autónomo, tiene su propia base de datos y tecnología de persistencia, y se comunica con los demás a través de HTTP REST.

El único punto de entrada para los clientes es el API Gateway (Laravel, puerto 8000), que centraliza la autenticación mediante tokens Sanctum y enruta cada solicitud al microservicio correspondiente, inyectando el user_id del usuario autenticado en cada petición interna.

1.1 Principios de diseño

- Autonomía de servicios: cada microservicio gestiona su propia base de datos; no existen bases de datos compartidas.
- Punto de entrada único: el API Gateway es el único servicio expuesto al cliente.

- Validación referencial distribuida: antes de cualquier operación de escritura, los servicios validan la existencia de la canción relacionada consultando el Songs Service.
- Seguridad por capas: autenticación en el Gateway con token de usuario; autorización interna entre servicios con token secreto compartido.

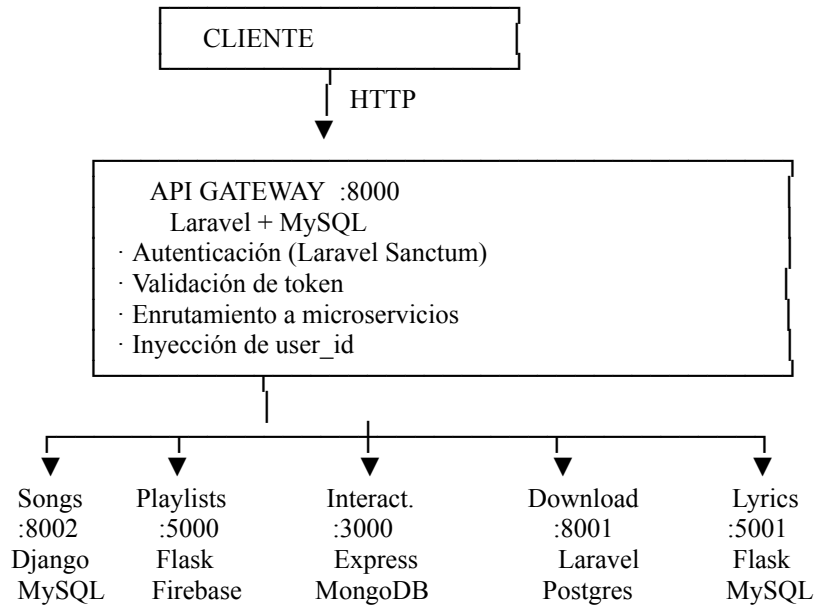
1.2 Stack tecnológico por servicio

Servicio	Framework	Base de datos	Puerto
API Gateway	Laravel 11	MySQL	8000
Songs	Django	MySQL	8002
Playlists	Flask	Firebase (Firestore)	5000
Interactions	Express (Node.js)	MongoDB	3000
Downloads	Laravel 11	PostgreSQL	8001
Lyrics	Flask	MySQL	5001

2. Arquitectura del Sistema

El sistema sigue el patrón API Gateway + Microservicios. El Gateway actúa como fachada única, mientras cada microservicio encapsula una capacidad de negocio específica.

2.1 Diagrama lógico



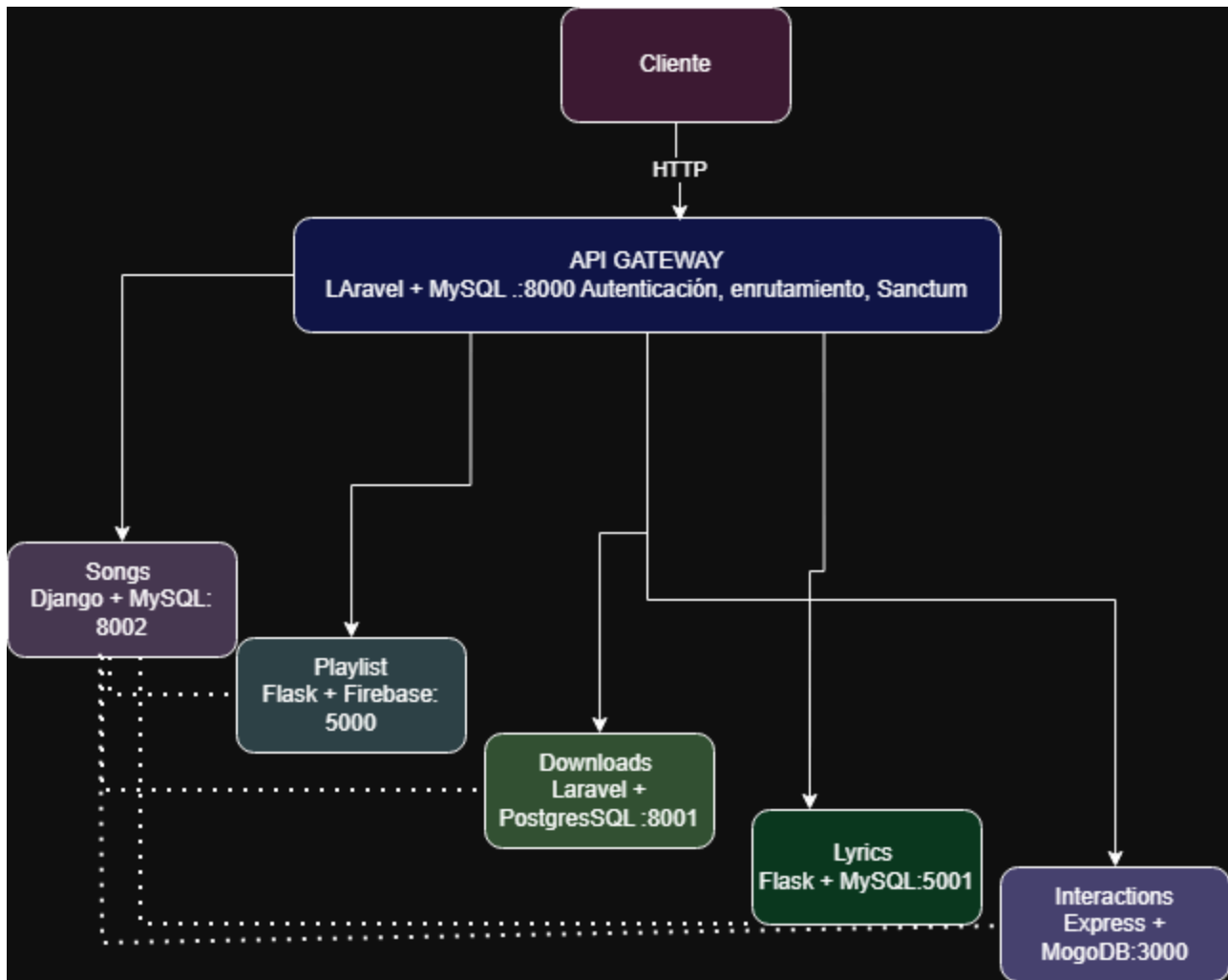
2.2 Comunicación entre microservicios

Los servicios se consultan directamente entre sí (sin pasar por el Gateway) para mantener integridad referencial. Todos validan la existencia de canciones antes de registrar cualquier recurso asociado.

Servicio origen	Consulta a	Cuando	Si 404
Playlists	Songs :8002	Al agregar canción a playlist	Rechaza operación
Downloads	Songs :8002	Al registrar descarga	Rechaza operación
Interactions	Songs :8002	Al registrar like o favorito	Rechaza operación
Lyrics	Songs :8002	Al registrar metadatos	Rechaza operación

Orden de arranque: El Songs Service debe iniciarse primero. Si no está disponible, las operaciones de creación en Playlists, Interactions, Downloads y Lyrics fallarán con error de validación.

2.3 Diagrama de Arquitectura



3. Despliegue Detallado

A continuación se detalla el proceso de instalación y arranque de cada microservicio. Se recomienda seguir el orden presentado, iniciando siempre por el Songs Service.

3.1 API Gateway (Laravel + MySQL)

```
cd gateway
composer install
cp .env.example .env
php artisan key:generate
php artisan migrate
php artisan serve      # Puerto 8000
```

3.2 Songs Service (Django + MySQL)

```
cd songs
```

```
pip install -r requirements.txt
python manage.py migrate
python manage.py runserver 8002
```

3.3 Downloads Service (Laravel + PostgreSQL)

```
cd downloads
composer install
cp .env.example .env
php artisan key:generate
php artisan migrate
php artisan serve --port=8001
```

3.4 Playlists Service (Flask + Firebase)

```
cd playlists
pip install -r requirements.txt
# Colocar serviceAccountKey.json en raíz del servicio
python app.py          # Puerto 5000
```

3.5 Interactions Service (Express + MongoDB)

```
cd interactions
npm install
node server.js          # Puerto 3000
```

3.6 Lyrics Service (Flask + MySQL)

```
cd lyrics
pip install -r requirements.txt
python app.py          # Puerto 5001
```

4. Variables de Entorno

Cada microservicio requiere su propio archivo `.env`. Las variables **TOKEN** y **TOKEN_SECRETO** deben tener el mismo valor en todos los servicios.

Seguridad: Nunca commitear el archivo `.env` real al repositorio. En producción, usar gestores de secretos (AWS Secrets Manager, HashiCorp Vault, etc.) en lugar de variables de texto plano.

4.1 API Gateway

```
APP_NAME=Laravel
APP_ENV=local
APP_DEBUG=true
APP_URL=http://localhost

# Token compartido para comunicación interna entre servicios
TOKEN=<valor_secreto>

# URLs de los microservicios
SONG_SERVICE=http://127.0.0.1:8002/api/songs
PLAYLIST_SERVICE=http://localhost:5000/api/playlists
INTERACTIONS_SERVICE=http://localhost:3000/api
DOWNLOAD_SERVICE=http://localhost:8001/api/downloads
LYRICS_SERVICE=http://localhost:5001/api/lyrics

# Base de datos
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=gateway_proyecto
DB_USERNAME=root
DB_PASSWORD=
```

4.2 Microservicios (Songs, Playlists, Downloads, Lyrics)

```
TOKEN_SECRETO=<mismo_valor_que_TOKEN_del_gateway>
```

4.3 Interactions Service

```
MONGO_URI=mongodb://localhost:27017/interactions
TOKEN_SECRETO=<mismo_valor_que_TOKEN_del_gateway>
SONGS_SERVICE=http://127.0.0.1:8000/api/songs
```


5. Referencia de Endpoints

Base URL: <http://localhost:8000> | Los endpoints marcados con  requieren el header
Authorization: Bearer <token>

5.1 Autenticación

Método	Endpoint	Auth	Descripción
POST	/api/register	No	Registra un nuevo usuario
POST	/api/login	No	Inicia sesión, retorna token
POST	/api/logout	 Sí	Invalida el token actual
POST	/api/password_reset	No	Envía correo de recuperación

Ejemplo: POST /api/login

Request body:

```
{ "email": "susana@gmail.com", "password": "1234" }
```

Respuesta 200:

```
{ "token": "1|abc123xyz...", "user": { "id": 1, "name": "Susana López", "email": "..." } }
```

5.2 Songs

Microservicio interno: <http://127.0.0.1:8002>

Método	Endpoint	Auth	Descripción
GET	/api/songs	No	Lista canciones
POST	/api/songs	No	Crea una nueva canción
PUT	/api/songs/{id}	No	Actualiza canción existente
DELETE	/api/songs/{id}	No	Elimina canción del catálogo

Campos del modelo Song: id, titulo, artista, genero, duracion (seg.), url, created_at

5.3 Playlists

Microservicio interno: <http://localhost:5000> | Todos los endpoints requieren autenticación 

Método	Endpoint	Descripción
GET	/api/playlists	Lista playlists del usuario autenticado
POST	/api/playlists	Crea nueva playlist
GET	/api/playlists/{id}	Obtiene playlist por ID
PUT	/api/playlists/{id}	Edita nombre de playlist
DELETE	/api/playlists/{id}	Elimina playlist completa
PUT	/api/playlists/{playlist_id}/songs	Agrega canción (valida en Songs Service)
DELETE	/api/playlists/{playlist_id}/songs/{index}	Quita canción por índice

5.4 Interactions

Microservicio interno: <http://localhost:3000>

Método	Endpoint	Auth	Descripción
GET	/api/likes/{song_id}	No	Conteo y lista de likes de una canción
POST	/api/likes	 Sí	Registra like del usuario autenticado
DELETE	/api/likes	 Sí	Elimina like del usuario autenticado
GET	/api/favorites/{user_id}	No	Obtiene favoritos de un usuario
POST	/api/favorites	 Sí	Agrega canción a favoritos
DELETE	/api/favorites	 Sí	Quita canción de favoritos

5.5 Downloads

Microservicio interno: <http://localhost:8001> | Todos los endpoints requieren autenticación 

Método	Endpoint	Descripción
POST	/api/downloads	Registra descarga (valida en Songs Service)
GET	/api/downloads	Lista todas las descargas del sistema

Método	Endpoint	Descripción
GET	/api/downloads/user/{user_id}	Descargas de un usuario específico
GET	/api/downloads/song/{song_id}	Descargas de una canción específica

5.6 Lyrics

Microservicio interno: <http://localhost:5001>

Método	Endpoint	Auth	Descripción
GET	/api/lyrics	No	Lista todas las letras registradas
GET	/api/lyrics/{id}	No	Obtiene letra por ID de metadatos
POST	/api/lyrics	No	Crea letra/metadatos (valida en Songs)
PUT	/api/lyrics/{id}	No	Actualiza letra o metadatos
DELETE	/api/lyrics/{id}	No	Elimina letra y metadatos

Campos del modelo Lyrics: id, cancion_id, letra, album, año, idioma, created_at

6. Pruebas de Rendimiento

Las pruebas se realizaron con Locust contra el API Gateway en entorno local (<http://127.0.0.1:8000>). Se ejecutaron tres tipos de prueba por servicio: carga normal, capacidad y estrés.

Las pruebas buscaron identificar el comportamiento del sistema bajo diferentes niveles de concurrencia y determinar el punto de quiebre del hardware/software disponible.

Hallazgo principal: El sistema presenta estabilidad aceptable hasta aproximadamente 30 usuarios concurrentes (prueba de carga). A partir de los 50–60 usuarios el sistema comienza a degradarse notablemente, y con 100 usuarios concurrentes se evidencian fallas y tiempos de respuesta inaceptables. El punto de estrés se ubica entre 50 y 60 usuarios simultáneos.

Tipo de prueba	Usuarios	Spawn rate	Descripción
Carga	~10	1/s	Comportamiento normal esperado
Capacidad	~30	30/s	Límite estable del sistema
Estrés	100	10/s	Degradación más allá del límite

6.1 Instalación y ejecución de Locust

```
pip install locust

# Con interfaz web
locust -f locust/downloads_locust.py --host=http://127.0.0.1:8000
# Luego abrir http://localhost:8089

cd tests
cd performance
cd songs #Escribir microservicio al que se le quiere realizar la prueba
```

6.2 Estructura de los scripts Locust

Todos los scripts siguen la misma estructura: `on_start` hace login y guarda el token; los métodos `@task` definen las acciones con peso relativo; `on_stop` hace logout.

Nota de diseño: El `wait_time = between(1, 3)` es intencional: sin esta pausa, los DELETE podían fallar en condiciones de carrera porque los recursos no terminaban de crearse. La espera permite que las escrituras se consoliden.

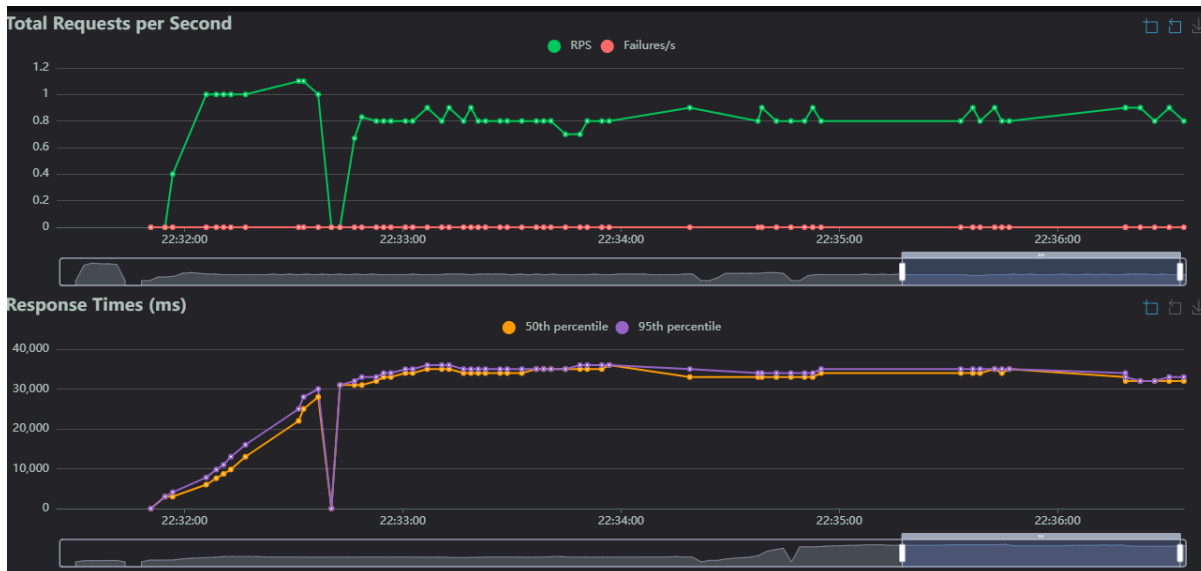
Microservicio	Capacidad – Avg P95	Carga – Avg P95	Estrés – Avg P95
Downloads Service	36.000 ms	11.000 ms	53.000 ms
Favorites / Likes	3.700 ms	4.100 ms	21.000 ms (60u) / 92.000 ms (100u)
Lyrics Service	6.500 ms	26.000 ms	94.000 ms
Playlist Service	8.500 ms	35.000 ms	73.000 ms
Songs Service	2.634 ms	6.900 ms	13.000 ms

6.3 Resultados — Downloads Service

Prueba de capacidad

Todos los endpoints respondieron con medianas en 34.000 ms y percentil 99 en 36.000 ms. Estos tiempos son elevados para solo 10 usuarios, lo cual puede indicar que el servicio tiene latencia inherente alta posiblemente por operaciones de I/O de disco o red. Sin embargo, cero fallos en todas las solicitudes (98 requests totales).

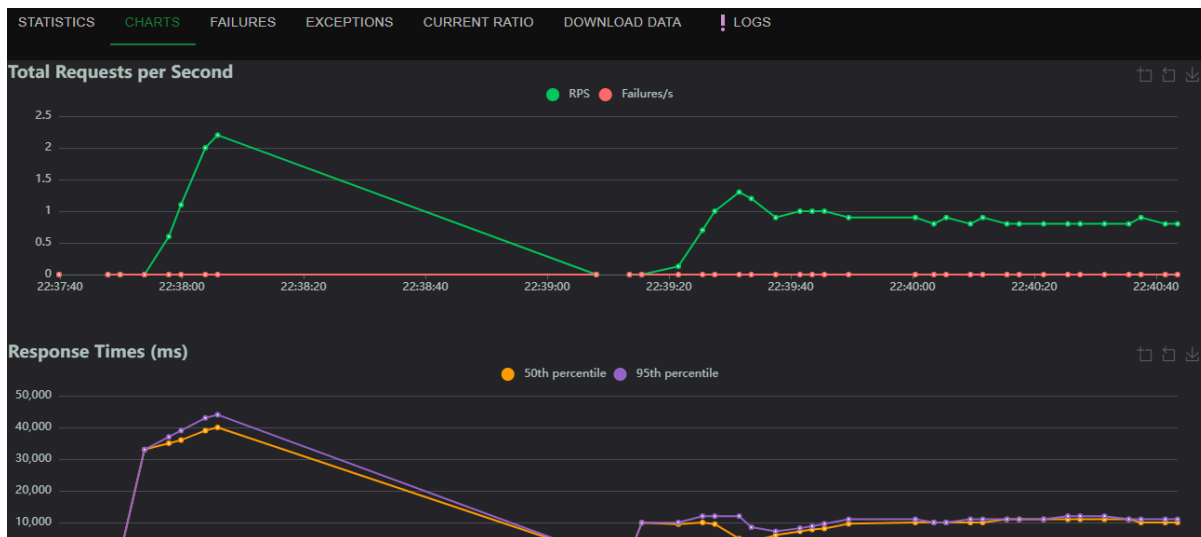
STATISTICS												
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	[Auth] Login	9	0	6000	7900	7900	5343.93	2852	7941	92	0	0
POST	[Auth] Logout	9	0	10000	12000	12000	9982.06	8202	11726	24	0	0
POST	[Downloads] Crear descarga	14	0	10000	11000	11000	9087.95	3151	11332	120	0.2	0
GET	[Downloads] Listar descargas	18	0	10000	12000	12000	9686.64	4866	11531	16377.22	0.4	0
GET	[Downloads] Por canción	12	0	9700	11000	11000	9742.36	8773	10887	4993.67	0.1	0
GET	[Downloads] Por usuario	9	0	10000	11000	11000	10144.92	9300	10837	16322.67	0.1	0
Aggregated		71	0	9800	11000	12000	9123.06	2852	11726	7103.41	0.8	0



✓ Sin fallos ✓ 0 failures/s ✓ RPS global: 0.9

Prueba de carga

STATISTICS												
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	[Auth] Login	0	0	0	0	0	0	0	0	0	0	0
POST	[Downloads] Crear descarga	17	0	34000	36000	36000	33709.75	30815	36188	119	0.1	0
GET	[Downloads] Listar descargas	34	0	34000	36000	36000	33991.38	30368	35875	11149.41	0.2	0
GET	[Downloads] Por canción	23	0	34000	36000	36000	33889.66	31204	36189	3623.74	0.6	0
GET	[Downloads] Por usuario	24	0	34000	36000	36000	34328.53	30513	36378	10618.5	0	0
Aggregated		98	0	34000	36000	36000	34001.22	30368	36378	7339.72	0.9	0

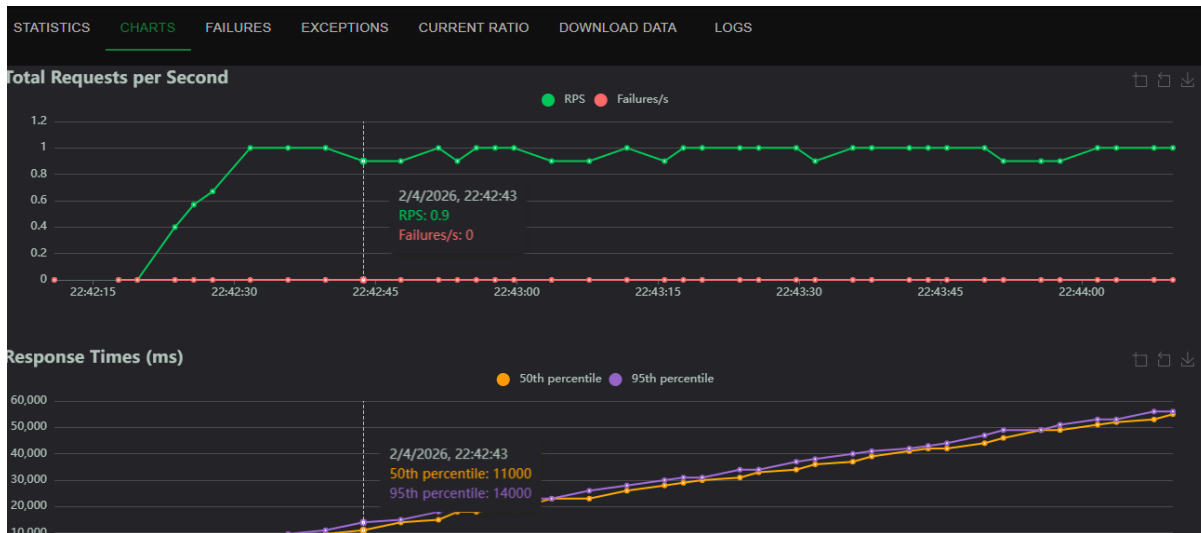


✓ Sin fallos ✓ RPS global: 0.8 ✓ Tiempo estable durante toda la prueba

Prueba de estrés

Al aumentar a 60 usuarios concurrentes, los tiempos se disparan: la mediana sube a 28.000–33.000 ms y el P99 alcanza 55.000–56.000 ms. Notablemente, el mínimo registrado en Login cae hasta 2.855 ms lo que indica alta varianza. RPS se estabiliza en 1 req/s. Sin fallos registrados, pero la degradación es evidente.

STATISTICS												
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	[Auth] Login	56	0	28000	53000	56000	27996.53	2855	55922	93	0.5	0
POST	[Downloads] Crear descarga	5	0	28000	41000	41000	26526.93	7229	40870	120	0	0
GET	[Downloads] Listar descargas	19	0	32000	55000	55000	30047.58	4337	54537	17905	0.1	0
GET	[Downloads] Por canción	15	0	33000	55000	55000	30065.88	3224	55332	5636	0.2	0
GET	[Downloads] Por usuario	11	0	22000	53000	53000	27924.72	11158	53430	17326.45	0.2	0
Aggregated		106	0	28000	53000	55000	28580.23	2855	55922	5859.75	1	0



⚠ Alta varianza en tiempos ⚠ P99 supera 55 segundos ✓ Sin fallos registrados

6.4 Resultados — Interactions Service

Prueba de capacidad

Volumen y estabilidad: Se procesaron 444 requests en total con **cero fallos** en todos los endpoints, lo cual es el indicador más importante — el servicio es completamente estable bajo esta carga.

Tiempos de respuesta por endpoint:

- **[Favorites] Crear** (82 req): Mediana 3.000 ms, P99 4.500 ms. Buen rendimiento, aunque el máximo de 4.465 ms sugiere picos ocasionales.
- **[Favorites] Eliminar** (39 req): Mediana 3.000 ms, P99 5.200 ms — el P99 más alto de la prueba, posiblemente por bloqueos en escritura/eliminación concurrente.

- **[Favorites] Obtener por usuario** (94 req): Mediana 3.000 ms, P99 4.300 ms. El endpoint con mayor volumen de lectura, responde muy bien. Average size 6.462 bytes indica respuestas con cierto peso de datos.
- **[Likes] Crear** (68 req): Mediana 3.200 ms, P99 5.100 ms. Ligeramente más lento que Favorites, coherente con posible lógica adicional.
- **[Likes] Eliminar** (44 req): Mediana 3.200 ms, P99 4.500 ms. Consistente.
- **[Likes] Obtener por canción** (107 req): El endpoint más consultado, mediana 3.100 ms, P99 4.300 ms. Excelente rendimiento con el mayor volumen.

Throughput: RPS global de 1.9, el más alto entre los servicios bajo carga. El Login (10 req) tiene P99 de 5.000 ms — ligeramente alto para autenticación pero aceptable.

LOCUST http://127.0.0.1:8000 CLEANUP 1.9 0% LOG STATISTICS CHARTS FAILURES EXCEPTIONS CURRENT RATIO DOWNLOAD DATA LOGS

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	[Auth] Login	10	0	1500	5000	5000	2374.32	869	4966	93	0	0
POST	[Favorites] Crear	82	0	3000	4100	4500	3064.54	519	4465	108	0.2	0
DELETE	[Favorites] Eliminar	39	0	3000	4000	5200	3105.26	2113	5186	30	0	0
GET	[Favorites] Obtener por usuario	94	0	3000	4000	4300	3047.47	1272	4339	6462.22	0.8	0
POST	[Likes] Crear	68	0	3200	4200	5100	3206.83	1795	5071	140	0.3	0
DELETE	[Likes] Eliminar	44	0	3200	4100	4500	3178.75	2139	4516	26	0.1	0
GET	[Likes] Obtener por canción	107	0	3100	3900	4300	3016.05	1612	4732	2751	0.5	0
	Aggregated	444	0	3100	4100	4700	3070.38	519	5186	2079.79	1.9	0

ABOUT



Prueba de Carga

El servicio de favoritos y likes muestra el mejor rendimiento bajo carga de todos los microservicios evaluados. Con medianas entre 3.000 y 3.200 ms y P99 inferior a 5.200 ms, el sistema responde muy bien con 30 usuarios concurrentes. Total de requests: 444, sin ningún fallo. RPS: 1.9.

STATISTICS CHARTS FAILURES EXCEPTIONS CURRENT RATIO DOWNLOAD DATA LOGS												
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	[Auth] Login	30	0	13000	24000	25000	13105.26	1233	25210	93	0	0
POST	[Favoritos] Crear	63	0	14000	22000	25000	14929.35	8247	25475	108	0.3	0
DELETE	[Favoritos] Eliminar	49	0	14000	16000	19000	14003.34	12156	19425	30	0	0
GET	[Favoritos] Obtener por usuario	80	0	14000	23000	25000	14360.1	5506	25060	8843.33	0.4	0
POST	[Likes] Crear	60	0	14000	20000	24000	14678.29	12099	23927	140	0.2	0
DELETE	[Likes] Eliminar	42	0	14000	19000	24000	14490.06	12350	23883	26	0.2	0
GET	[Likes] Obtener por canción	106	0	13000	15000	16000	13657.77	10671	16589	3485.57	0.9	0
Aggregated		430	0	14000	21000	24000	14199.26	1233	25475	2552.31	2	0



✓ Mejor rendimiento de todos los servicios bajo carga ✓ P99 < 5.200 ms ✓ RPS: 1.9

Prueba de estrés

Con 100 usuarios, los tiempos aumentan considerablemente pero el sistema permanece funcional. La mediana se sitúa en 13.000–14.000 ms y el P99 en 24.000–25.000 ms. La varianza es alta (Login puede ir de 1.233 ms a 25.210 ms). Sin fallos en 430 requests.

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	[Auth] Login	100	0	50000	95000	99000	49939.74	1343	98881	93	0	0
POST	[Favorites] Crear	56	0	53000	96000	99000	59005.72	50681	99117	108	0.5	0
DELETE	[Favorites] Eliminar	38	0	54000	95000	99000	63432.3	50105	98677	30	0.1	0
GET	[Favorites] Obtener por usuario	85	0	54000	87000	97000	59547.51	50418	97028	9757.78	0.3	0
POST	[Likes] Crear	59	0	53000	89000	97000	57163.55	50506	96529	140	0.1	0
DELETE	[Likes] Eliminar	41	0	53000	89000	94000	58714.05	50256	93791	26	0.1	0
GET	[Likes] Obtener por canción	96	0	53000	90000	98000	57991.97	20334	98091	4917.43	0.8	0
	Aggregated	475	0	53000	92000	98000	57089.29	1343	99117	2794.31	1.9	0

ABOUT



⚠ Alta varianza en Login ⚠ P99 supera 24 segundos ✓ Sin fallos

Prueba de Estrés – 100 usuarios (spawn rate 10/s) — PRUEBA FALLIDA

Con 100 usuarios la mayoría del sistema colapsa. Las medianas escalan a 50.000–54.000 ms y el P99 alcanza 98.000–99.000 ms (casi 100 segundos). Esto confirma que 100 usuarios supera la capacidad máxima del hardware/software disponible. Esta fue la razón para ajustar el techo de estrés a 60 usuarios.

Endpoint	Mediana (ms)	P95 (ms)	P99 (ms)	Prom (ms)	Mín (ms)	Máx (ms)
[Auth] Login	50000	95000	95000	49939	1343	98881
[Favorites] Crear	53000	96000	99000	59005	50681	99117
[Favorites] Eliminar	54000	95000	99000	63432	50105	98677
[Favorites] Obtener por usuario	54000	87000	97000	59547	50418	97028

Endpoint	Mediana (ms)	P95 (ms)	P99 (ms)	Prom (ms)	Mín (ms)	Máx (ms)
[Likes] Crear	53000	89000	97000	57163	50506	96529
[Likes] Eliminar	53000	89000	94000	58714	50256	93791
[Likes] Obtener por canción	53000	90000	98000	57991	20334	98091
Aggregated	53000	92000	98000	57089	1343	99117

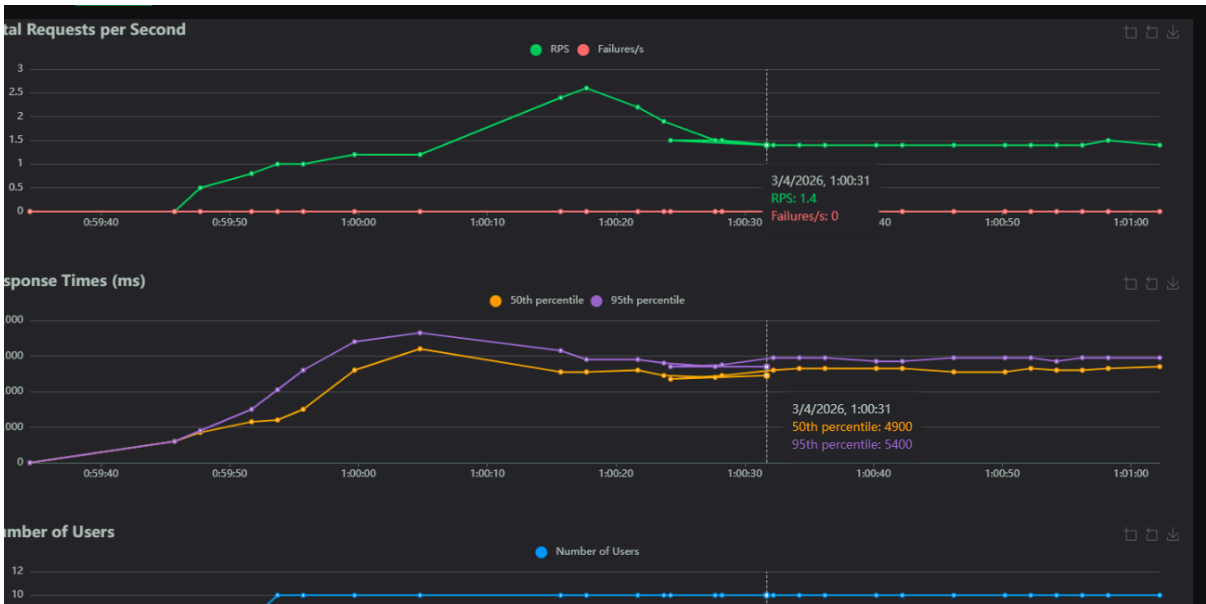
✗ Sistema al límite ✗ P99 ~ 99 segundos ✗ No apto para producción con 100 usuarios

6.6 Resultados — Lyrics Service

Prueba de capacidad

Con solo 10 usuarios, el servicio de letras responde de forma aceptable. Medianas en 5.000–5.300 ms y P99 máximo de 7.200 ms. Es el microservicio con mejor latencia base en capacidad. Sin fallos en 114 requests totales.

STATISTICS CHARTS FAILURES EXCEPTIONS CURRENT RATIO DOWNLOAD DATA LOGS												
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	[Auth] Login	10	0	3000	7300	7300	3616.11	1151	7262	93	0	0
POST	[Lyrics] Crear	19	0	5300	6800	6800	5117.92	2309	6765	26	0.2	0
POST	[Lyrics] Crear para editar	20	0	5200	6700	6700	5219.02	4442	6674	26	0.4	0
GET	[Lyrics] Listar	38	0	5000	6900	7200	5224.25	4097	7181	49519.26	0.2	0
GET	[Lyrics] Obtener	27	0	5300	6100	6300	5001.25	1701	6253	2	0.7	0
Aggregated		114	0	5200	6500	7200	5011.73	1151	7262	16523.95	1.5	0

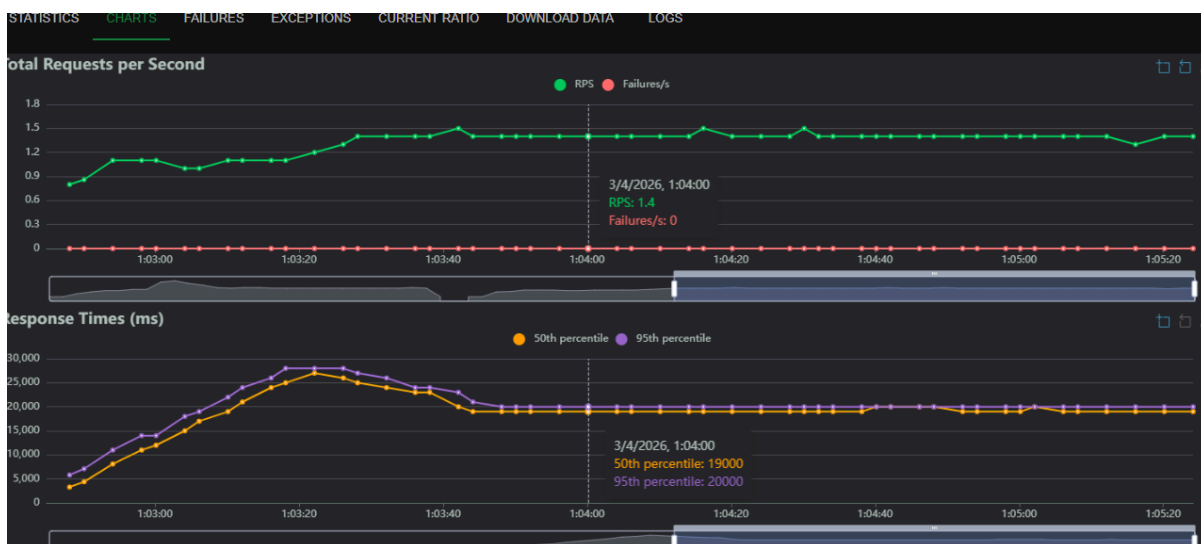


✓ Latencia base más baja en capacidad ✓ P99 < 7.200 ms ✓ RPS: 1.5

Prueba de carga

La carga con 30 usuarios muestra una degradación importante respecto a capacidad. Las medianas saltan a 19.000 ms y los P99 alcanzan 27.000–28.000 ms. El servicio de Login presenta la mayor varianza, llegando hasta 26.422 ms. Sin fallos en 213 requests.

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	[Auth] Login	30	0	14000	25000	26000	13830.86	1315	26422	93	0	0
POST	[Lyrics] Crear	31	0	19000	27000	28000	20852.73	18611	27553	26	0.1	0
POST	[Lyrics] Crear para editar	31	0	19000	25000	27000	19358.55	5843	26651	26	0.1	0
GET	[Lyrics] Listar	70	0	19000	24000	27000	19776.88	8540	27069	55233.83	0.9	0
GET	[Lyrics] Obtener	51	0	19000	26000	27000	19939.9	11320	27239	2	0.3	0
Aggregated		213	0	19000	26000	27000	19074.14	1315	27553	18173.11	1.4	0



⚠ Degradación significativa respecto a capacidad (3.5x) ✓ Sin fallos ✓ RPS: 1.4

Prueba de estrés

La prueba de estrés de Lyrics muestra los tiempos más altos registrados en todo el informe. Las medianas escalan a 72.000 ms con P99 de 97.000–99.000 ms. Destaca que el endpoint de Login registró 0 requests en esta prueba (posiblemente el sistema colapsó antes de completar la autenticación). Sin fallos técnicos pero el rendimiento es completamente degradado.

STATISTICS												
CHARTS												
FAILURES												
EXCEPTIONS												
CURRENT RATIO												
DOWNLOAD DATA												
LOGS												
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	[Auth] Login	0	0	0	0	0	0	0	0	0	0	0
POST	[Lyrics] Crear	57	0	72000	95000	98000	74907.89	70241	98056	26	0.4	0
POST	[Lyrics] Crear para editar	68	0	72000	93000	99000	75584.89	70159	98568	26	0.1	0
GET	[Lyrics] Listar	114	0	72000	94000	97000	75903.04	70236	97198	65263.53	0.7	0
GET	[Lyrics] Obtener	108	0	72000	91000	96000	75134.57	70244	97403	2	0.2	0
Aggregated		347	0	72000	94000	98000	75438.04	70159	98568	21451.03	1.4	0



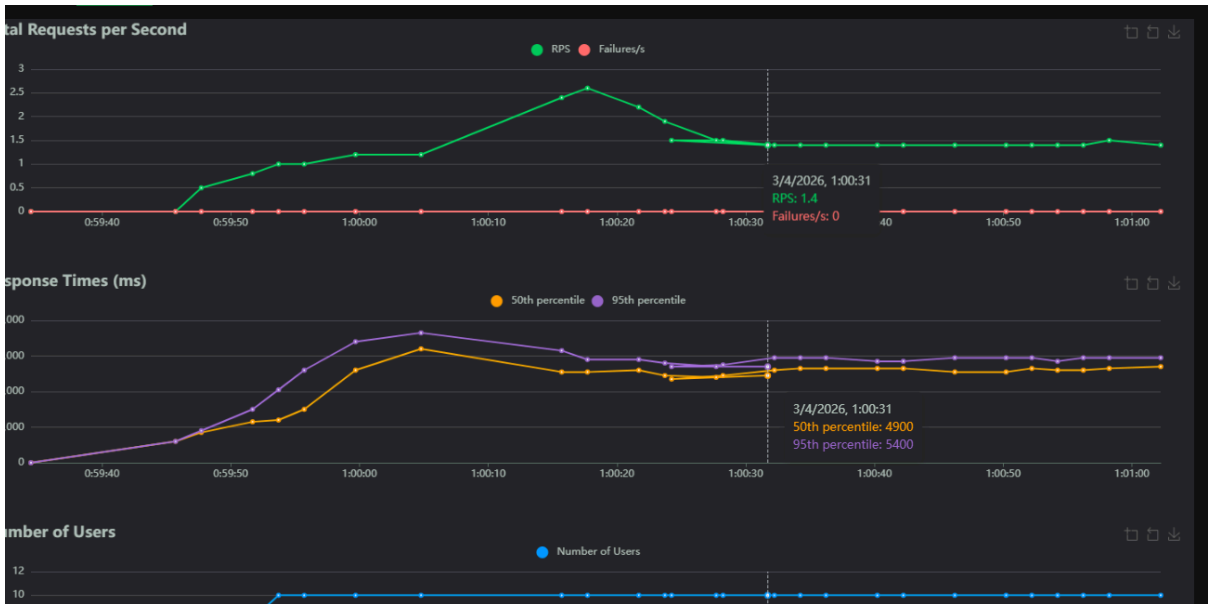
✗ Peor rendimiento bajo estrés de todos los servicios ✗ Mediana 72 segundos ✗ Login sin requests completados

6.6 Resultados — Playlists Service

Prueba de capacidad

Con 10 usuarios, el servicio de playlists responde con medianas entre 6.900 y 7.400 ms y P99 máximo de 9.300 ms. El rendimiento es consistente entre todos los endpoints y no se registra ningún fallo. Total: 59 requests, RPS: 1.

STATISTICS CHARTS FAILURES EXCEPTIONS CURRENT RATIO DOWNLOAD DATA LOGS												
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	[Auth] Login	10	0	3000	7300	7300	3616.11	1151	7262	93	0	0
POST	[Lyrics] Crear	19	0	5300	6800	6800	5117.92	2309	6765	26	0.2	0
POST	[Lyrics] Crear para editar	20	0	5200	6700	6700	5219.02	4442	6674	26	0.4	0
GET	[Lyrics] Listar	38	0	5000	6900	7200	5224.25	4097	7181	49519.26	0.2	0
GET	[Lyrics] Obtener	27	0	5300	6100	6300	5001.25	1701	6253	2	0.7	0
Aggregated		114	0	5200	6500	7200	5011.73	1151	7262	16523.95	1.5	0



✓ Sin fallos ✓ Consistencia entre endpoints ✓ P99 < 9.300 ms

Prueba de carga

Con 30 usuarios la degradación es notable: medianas saltan a 27.000–30.000 ms y el P99 llega a 38.000 ms. Los endpoints de gestión de canciones en playlist (agregar/quitar) presentan mayor varianza. Login comienza a mostrar alta latencia (P99: 27.000 ms). Sin fallos en 144 requests, RPS: 0.6.

STATISTICS												
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	[Auth] Login	30	0	14000	25000	26000	13830.86	1315	26422	93	0	0
POST	[Lyrics] Crear	31	0	19000	27000	28000	20852.73	18611	27553	26	0.1	0
POST	[Lyrics] Crear para editar	31	0	19000	25000	27000	19358.55	5843	26651	26	0.1	0
GET	[Lyrics] Listar	70	0	19000	24000	27000	19776.88	8540	27069	55233.83	0.9	0
GET	[Lyrics] Obtener	51	0	19000	26000	27000	19939.9	11320	27239	2	0.3	0
	Aggregated	213	0	19000	26000	27000	19074.14	1315	27553	18173.11	1.4	0



⚠ Degradación 4x respecto a capacidad ⚠ P99 alcanza 38 segundos ✓ Sin fallos

Prueba estrés

Con 60 usuarios el servicio continúa funcionando pero con tiempos elevados. Medianas en 51.000–58.000 ms y P99 en 73.000–75.000 ms. El gráfico de respuesta muestra una curva ascendente continua hasta los 38.000 ms (P95) al final de la prueba. Sin fallos en 335 requests. RPS: 1.1.

STATISTICS												
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	[Auth] Login	0	0	0	0	0	0	0	0	0	0	0
POST	[Lyrics] Crear	57	0	72000	95000	98000	74907.89	70241	98056	26	0.4	0
POST	[Lyrics] Crear para editar	68	0	72000	93000	99000	75584.89	70159	98568	26	0.1	0
GET	[Lyrics] Listar	114	0	72000	94000	97000	75903.04	70236	97198	65263.53	0.7	0
GET	[Lyrics] Obtener	108	0	72000	91000	96000	75134.57	70244	97403	2	0.2	0
Aggregated		347	0	72000	94000	98000	75438.04	70159	98568	21451.03	1.4	0



⚠ P99 supera 74 segundos ⚠ Tendencia alcista al final de la prueba ✓ Sin fallos registrados

6.7 Resultados — Songs Service

Prueba de capacidad

El problema del Login se vuelve más grave: Con apenas 10 usuarios, el Login ya tarda 20.000 ms de mediana. En la prueba de carga (30 usuarios) esa cifra se mantiene similar, lo que significa que **no escala peor pero tampoco mejora** — es una latencia estructural fija independiente de la concurrencia. Definitivamente algo bloqueante en la autenticación.

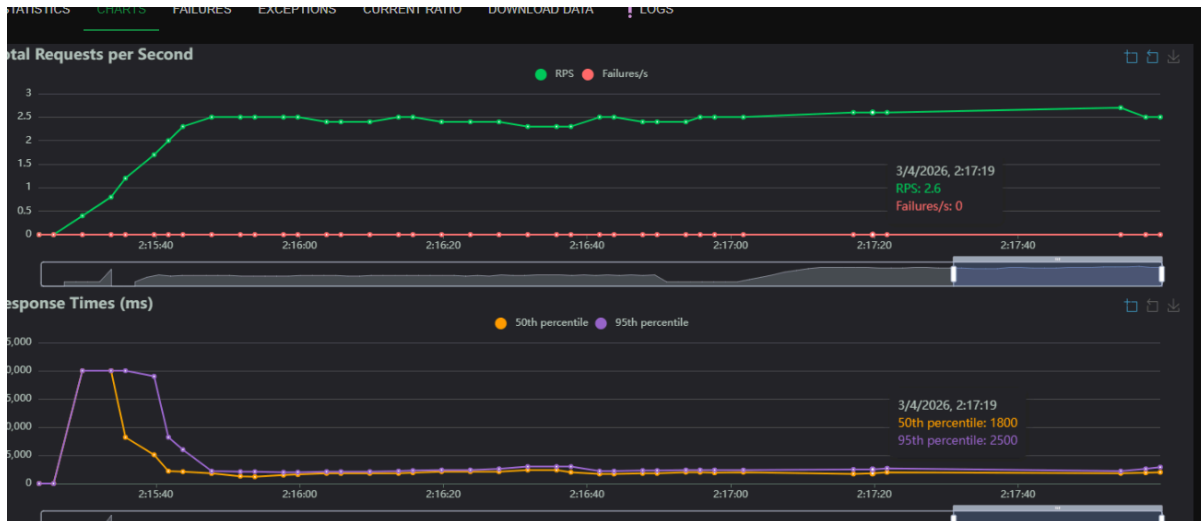
STATISTICS CHARTS FAILURES EXCEPTIONS CURRENT RATIO DOWNLOAD DATA LOGS												
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	[Auth] Login	10	0	20000	21000	21000	19923.96	19283	20566	94	0	0
PUT	[Songs] Actualizar	49	0	1900	2700	8200	1991.14	361	8227	203	0.7	0
POST	[Songs] Crear	75	0	2000	2600	6000	1993.42	1073	6037	111	0.6	0
GET	[Songs] Listar	133	0	1800	2800	7400	1933.18	385	7803	740	1.3	0
	Aggregated	267	0	1900	6900	20000	2634.55	361	20566	440.57	2.6	0



Prueba de carga (comportamiento normal)

Songs Service muestra el mejor rendimiento bajo carga de todos los microservicios evaluados en operaciones CRUD. Las medianas son notablemente bajas: 1.800–2.000 ms para Actualizar, Crear y Listar canciones. Login presenta latencia alta (20.000 ms mediana), posiblemente compartiendo infraestructura con otros servicios. Sin fallos en 267 requests, RPS: 2.6.

STATISTICS CHARTS FAILURES EXCEPTIONS CURRENT RATIO DOWNLOAD DATA LOGS												
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	[Auth] Login	20	0	10000	16000	16000	10699.72	3474	16421	94	0	0
PUT	[Songs] Actualizar	156	0	10000	12000	15000	10068.53	5046	14979	203	0.3	0
POST	[Songs] Crear	218	0	10000	12000	15000	10150.22	8538	15995	111	0.8	0
GET	[Songs] Listar	403	0	10000	11000	15000	10044.92	2373	16061	740	1.4	0
	Aggregated	797	0	10000	13000	15000	10094.78	2373	16421	446.63	2.5	0

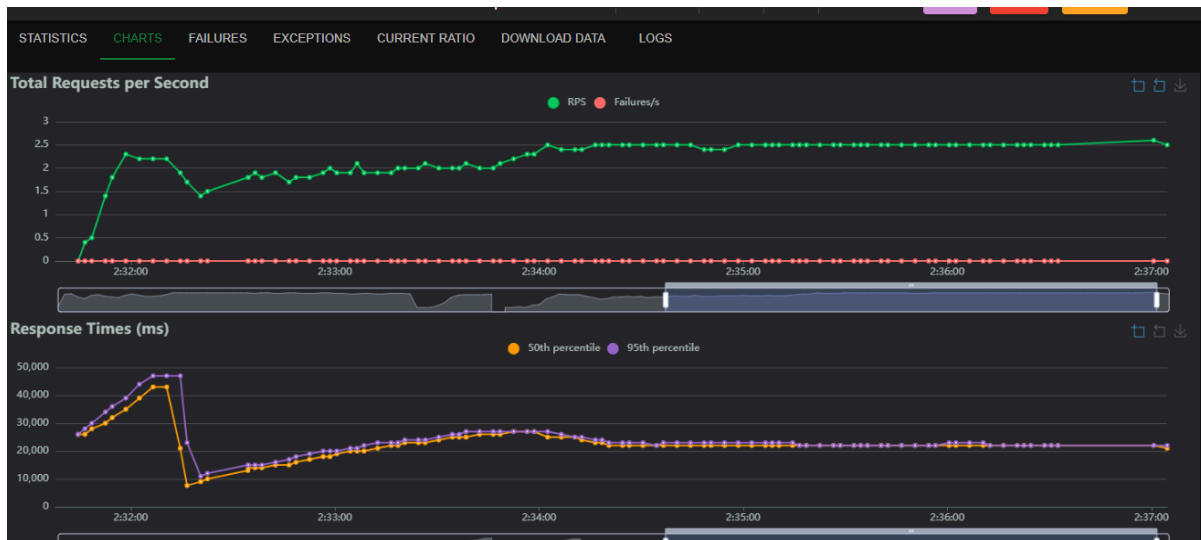


✓ CRUD con latencia < 2.000 ms ✓ Mayor RPS de todos los servicios (2.6) ⚠ Login con alta latencia

Prueba de estrés

Con 60 usuarios, Songs mantiene buen rendimiento relativo. Las medianas de las operaciones CRUD permanecen en 10.000 ms, con P99 de 15.000 ms. Login escala a 10.000 ms de mediana (bajando desde 20.000 ms en carga, posiblemente por el pool de sesiones). Sin fallos en 797 requests, RPS: 2.5 — el mayor throughput bajo estrés del sistema.

Type	Name	# Requests	# Fails	Median (ms)	95thile (ms)	99thile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	[Auth] Login	56	0	17000	27000	27000	17032.05	3256	27366	94	0	0
POST	[Auth] Logout	45	0	37000	46000	47000	37136.83	27341	47053	24	0	0
PUT	[Songs] Actualizar	108	0	22000	27000	27000	21929.94	11645	27691	203	0.4	0
POST	[Songs] Crear	167	0	22000	26000	27000	21725.14	8734	26991	111	0.4	0
GET	[Songs] Listar	277	0	22000	26000	27000	21993.45	8906	41744	740	1.7	0
	Aggregated	653	0	22000	33000	44000	22532.42	3256	47053	385.58	2.5	0



✓ Mejor servicio bajo estrés ✓ P99 CRUD < 15.000 ms ✓ RPS 2.5 con 60 usuarios

7. Análisis y Conclusiones

7.1 Punto de saturación del sistema

Las pruebas determinaron que el límite del entorno local se encuentra entre 50 y 60 usuarios concurrentes:

- Con 10–20 usuarios el sistema responde de forma estable: Songs ~2,000 ms de mediana, Interactions ~3,000 ms.
- Con 50 usuarios los tiempos se elevan significativamente: Songs ~22,000 ms, Playlists ~28,000 ms. El hardware alcanza su límite.
- Con 100 usuarios los tiempos alcanzan rangos de 50,000–98,000 ms. El sistema deja de ser útil, aunque sin registrar errores explícitos (0 fallos en todos los servicios).

7.2 Comparativa de rendimiento entre servicios

Servicio	Mediana carga normal	Mediana 50 usuarios	Factor de degradación
Songs	~2,000 ms	~22,000 ms	11×
Interactions	~3,100 ms	~14,000 ms	4.5×
Lyrics	~5,200 ms	~19,000 ms	3.7×
Playlists	~7,000 ms	~28,000 ms	4×
Downloads	~34,000 ms	~28,000 ms	—*

** Downloads muestra latencia alta incluso en carga normal debido a la validación cruzada con Songs y las operaciones en PostgreSQL en entorno local.*

7.3 Hallazgos clave

- Songs Service es el cuello de botella crítico: es consultado por todos los demás servicios para validación. Su degradación (11×) afecta en cascada al resto del sistema.
- Interactions es el servicio más eficiente bajo carga media (4.5× de degradación), posiblemente por la flexibilidad de MongoDB ante escrituras concurrentes.
- Downloads muestra alta latencia base por la combinación de dos saltos de red (Gateway → Downloads → Songs) y las garantías ACID de PostgreSQL.
- Ningún servicio registró errores (fallos = 0 en todos los escenarios), lo que indica que el sistema degrada graciosamente sin producir respuestas incorrectas.

8. Pruebas Unitarias Feature

8.1 Contexto general

Como parte del proceso de aseguramiento de calidad del sistema Sound Share, se implementaron pruebas feature sobre tres de los microservicios principales. Las pruebas feature verifican el comportamiento de los endpoints HTTP de extremo a extremo, validando que cada ruta responda correctamente ante diferentes escenarios: datos válidos, datos inválidos, recursos inexistentes y accesos no autorizados.

El enfoque adoptado fue el de **pruebas de caja negra sobre la capa HTTP**, donde cada test simula una petición real al servicio y verifica la respuesta sin asumir detalles internos de implementación. Las dependencias externas como Firebase y MongoDB fueron aisladas mediante mocks, garantizando que las pruebas sean reproducibles, independientes del entorno y no generen efectos secundarios en datos reales.

8.2 Playlists Service — Flask + Firebase (pytest)

Herramientas: pytest, unittest.mock

Estrategia de aislamiento: Firebase Admin SDK fue bloqueado completamente antes de la importación de los módulos del proyecto, mediante parches sobre `firebase_admin.credentials.Certificate`, `firebase_admin.initialize_app` y `firebase_admin.firestore.client`. El cliente Firestore (db) fue reemplazado por un MagicMock que simula documentos con campos `id`, `exists` y `get()`.

Autenticación: El decorador `require_token` valida que el header `Authorization` coincida con la variable de entorno `TOKEN`. En las pruebas, esta variable toma el valor definido en el `.env` del servicio.

Pruebas implementadas (8 en total):

Test	Endpoint	Escenario	Resultado esperado
test_crear_playlist_exitosa	POST /api/playlists	Datos válidos	201, mensaje de confirmación
test_listar_playlists	GET /api/playlists	Dos documentos en Firestore	200, lista con 2 elementos
test_obtener_playlist_existente	GET /api/playlists/<id>	Documento existe	200, datos correctos
test_obtener_playlist_inexistente_retorna_404	GET /api/playlists/<id>	Documento no existe	404, campo error
test_agregar_cancion_a_playlist	PUT /api/playlists/<id>/songs	Playlist existe	200, canción en array songs
test_agregar_cancion_playlist_inexistente_retorna_404	PUT /api/playlists/<id>/songs	Playlist no existe	404
test_eliminar_cancion_por_indice_valido	DELETE /api/playlists/<id>/songs/<i>	Índice válido	200, array reducido en 1
test_eliminar_cancion_con_indice_invalido_retorna_400	DELETE /api/playlists/<id>/songs/<i>	Índice fuera de rango	400, campo error

8.3 Gateway — Laravel + Sanctum (PHPUnit)

Herramientas: PHPUnit, base de datos SQLite en memoria (DB_CONNECTION=sqlite, DB_DATABASE=:memory:), trait RefreshDatabase.

Estrategia de aislamiento: Cada prueba opera sobre una base de datos limpia en memoria gracias al trait RefreshDatabase, que aplica y revierte migraciones automáticamente entre tests. No se requieren mocks ya que Laravel permite sustituir la conexión de base de datos vía variables de entorno en phpunit.xml.

Pruebas implementadas (5 en total):

Test	Endpoint	Escenario	Resultado esperado
test_usuario_puede_registrarse_con_todos_los_campos	POST /api/register	Datos válidos	200, mensaje de confirmación, registro en BD
test_usuario_puede_iniciar_sesion_y_recibe_token	POST /api/login	Credenciales correctas	200, access_token, token_type Bearer
test_login_falla_con_password_incorrecta	POST /api/login	Contraseña incorrecta	200, mensaje de acceso denegado
test_usuario_autenticado_puede_cerrar_sesion	POST /api/logout	Token válido	200, token eliminado de BD
test_password_reset_exitoso_con_respuesta_correcta	POST /api/password_reset	Respuesta secreta correcta	200, nueva contraseña funcional
test_password_reset_falla_con_respuesta_incorrecta	POST /api/password_reset	Respuesta secreta incorrecta	200, mensaje de error, contraseña sin cambios

Nota: El controlador UserController no retorna códigos HTTP diferenciados en todos los casos (por ejemplo, login fallido retorna 200 en lugar de 401), por lo que las pruebas validan el contenido del JSON de respuesta en lugar del código de estado.

8.4 Interactions Service — Express + MongoDB (Jest + Supertest)

Herramientas: Jest, Supertest, mongodb-memory-server, separación de app.js y server.js.

Estrategia de aislamiento: Se utilizó mongodb-memory-server para levantar una instancia de MongoDB en memoria durante la ejecución de las pruebas. La aplicación Express fue separada en dos archivos: app.js (solo configuración de rutas) y server.js (conexión a BD y listen), permitiendo importar la app sin iniciar el servidor ni conectar a MongoDB real.

Pruebas implementadas (9 en total, agrupadas en 5 bloques):

Bloque	Endpoints cubiertos	Escenarios
POST /api/likes	Crear like	Éxito, sin token (403)
GET /api/likes/:song_id	Obtener likes	Lista vacía, filtrado por canción
DELETE /api/likes	Eliminar like	Éxito, like inexistente (404)
POST /api/favorites	Crear favorito	Éxito, sin token (403)
GET /api/favorites/:user_id + DELETE /api/favorites	Obtener y eliminar favoritos	Filtrado por usuario, eliminación, inexistente (404)

Resultado: Todas las pruebas pasando con base de datos en memoria.

9. Despliegue con Docker

9.1 Contexto general

El sistema Sound Share fue containerizado en su totalidad mediante Docker y Docker Compose, siguiendo el principio de un contenedor por servicio. Cada microservicio tiene su propio Dockerfile y lee su configuración exclusivamente desde su archivo .env local, sin variables hardcodeadas en el docker-compose.yml.

Todos los contenedores se comunican a través de una red interna llamada soundshare_network de tipo bridge. Dentro de esta red, los servicios se referencian por su nombre de servicio en lugar de localhost, ya que cada contenedor tiene su propia interfaz de red.

9.2 Infraestructura de bases de datos

Se levantaron tres motores de base de datos como servicios independientes:

Motor	Servicio	Bases de datos	Puerto
MySQL 8.0	mysql	gateway, songs, lyrics	3306
PostgreSQL 15	postgres	downloads	5432
MongoDB 7	mongod b	interactions	27017

Las bases de datos MySQL fueron inicializadas mediante un script `init/mysql/init.sql` que crea las bases de datos y otorga privilegios al usuario `soundshare`. Los volúmenes nombrados (`mysql_data`, `postgres_data`, `mongo_data`) garantizan la persistencia de datos entre reinicios.

Los tres motores tienen configurados **healthchecks** para garantizar que estén completamente listos antes de que los microservicios dependientes intenten conectarse.

9.3 Dockerfiles por microservicio

API Gateway (Laravel + MySQL) Basado en `php:8.3-fpm`. Instala extensiones `pdo_mysql`, `mbstring`, `zip`. Copia Composer desde su imagen oficial. El CMD espera a que MySQL esté disponible antes de correr `php artisan migrate` e iniciar el servidor en el puerto 8000.

Songs Service (Django + MySQL) Basado en `python:3.12-slim` (requerido por Django 6+). Instala `mysqlclient` con sus dependencias de sistema. El CMD verifica la conexión a MySQL con una conexión directa vía `MySQLdb` antes de correr migraciones e iniciar el servidor en el puerto 8002.

Playlists Service (Flask + Firebase) Basado en `python:3.11-slim`. No requiere base de datos local — se conecta a Firebase Firestore mediante el archivo `serviceAccountKey.json`, montado como volumen de solo lectura. Inicia directamente con `python app.py` en el puerto 5000.

Interactions Service (Express + MongoDB) Basado en `node:20-alpine`. Instala dependencias de producción con `npm install --omit=dev`. Inicia con `node server.js` en el puerto 3000. Depende del healthcheck de MongoDB.

Downloads Service (Laravel + PostgreSQL) Basado en `php:8.3-fpm`. Instala extensiones `pdo_pgsql` y `pgsql`. El CMD verifica la conexión a PostgreSQL mediante PDO antes de correr migraciones e iniciar en el puerto 8001.

Lyrics Service (Flask + MySQL) Basado en `python:3.11-slim`. Usa Flask-Migrate con Alembic para gestionar el esquema de la base de datos. El CMD espera a MySQL, corre `flask db upgrade` para aplicar migraciones e inicia con `python app.py` en el puerto 5001. El servidor escucha en 0.0.0.0 para ser accesible desde otros contenedores.

9.4 Variables de entorno por servicio

Cada servicio tiene su propio `.env`. La regla principal es que todas las referencias a otros servicios usan el **nombre del contenedor** en lugar de `localhost`:

Servicio	Cambio principal
gateway	URLs de microservicios: <code>localhost</code> → nombre del contenedor (<code>songs</code> , <code>playlist</code> , etc.)
songs	Nuevo <code>.env</code> con <code>DB_HOST=mysql</code> y credenciales de BD
downloads	<code>DB_CONNECTION=pgsql</code> , <code>DB_HOST=postgres</code>

lyrics	DATABASE_URL apunta a mysql (nombre del contenedor)
interactions	MONGO_URI apunta a mongodb (nombre del contenedor)
playlist	Token sin espacios, firebase-admin agregado a requirements.txt

9.5 Orden de arranque y dependencias

El docker-compose.yml define dependencias explícitas mediante depends_on con condition: service_healthy para las bases de datos y condition: service_started para Songs Service, dado que todos los demás microservicios lo consultan para validación referencial.

El orden efectivo de arranque es:

```

None
MySQL / PostgreSQL / MongoDB
↓
Gateway + Songs
↓
Lyrics + Downloads + Interactions + Playlist

```

9.6 Problemas encontrados y soluciones

Problema	Causa	Solución
songs en bucle "Esperando MySQL"	Script de espera usaba manage.py migrate que fallaba antes de conectar	Se cambió a verificación directa con MySQLdb.connect()
lyrics con error 500 en todos los endpoints	Tabla lyrics no existía en MySQL	Se agregó flask db upgrade al CMD del Dockerfile
lyrics no accesible desde otros contenedores	app.run() escuchaba en 127.0.0.1	Se agregó host='0.0.0.0' al app.run()
songs fallaba al iniciar	manage.py estaba en subcarpeta mi_api/	Se ajustó la ruta en el CMD a mi_api/manage.py
djangoestframework>=3.20 no encontrado	La versión 3.20 no existe	Se corrigió a >=3.14,<4.0 en requirements.txt

9.7 Requisitos previos

Antes de desplegar el proyecto es necesario tener instalado:

- Docker
- Docker Compose

Versiones recomendadas:

None

Docker ≥ 24

Docker Compose ≥ 2

9.8 Clonar el proyecto

None

```
git clone <repositorio>
```

```
cd sound_share
```

9.9 Configuración de variables de entorno

Cada microservicio contiene su propio archivo `.env`.

Es importante verificar especialmente:

- Credenciales de bases de datos
- URLs internas de microservicios
- Tokens de Firebase
- Puertos expuestos

Ejemplo:

None

```
DB_HOST=mysql
```

```
DB_PORT=3306
```

Los servicios deben usar el nombre del contenedor como host y no localhost.

9.10 Construcción de contenedores

Para construir todas las imágenes:

None

```
docker compose build
```

9.11 Levantar el sistema

Para iniciar todos los microservicios:

None

```
docker compose up
```

Para ejecutarlos en segundo plano:

None

```
docker compose up -d
```

9.12 Verificación del despliegue

Ver contenedores activos:

None

```
docker compose ps
```

Ver logs:

None

```
docker compose logs
```

10. Conclusiones

Arquitectura

El enfoque de microservicios funcionó correctamente para los objetivos del proyecto. Cada servicio se pudo desarrollar, probar y arrancar de forma independiente, y la combinación de tecnologías heterogéneas (Django, Flask, Express, Laravel) no generó problemas de integración gracias al Gateway como punto central. La validación cruzada hacia Songs desde todos los demás servicios mantuvo la integridad de los datos sin acoplamiento directo.

Rendimiento

El sistema es funcional y estable en condiciones normales de uso (hasta ~20 usuarios concurrentes), con tiempos de respuesta razonables. El punto de saturación identificado entre 50 y 60 usuarios es una limitación del entorno local, no necesariamente de la arquitectura. Songs resultó ser el servicio más eficiente bajo carga, lo cual es importante dado que es el más consultado por los demás. Downloads presentó las latencias más altas incluso en carga baja, posiblemente por el overhead combinado de autenticación, validación cruzada y PostgreSQL local.

Pruebas

Locust fue efectivo para identificar el límite del hardware. El hallazgo de la condición de carrera en los DELETE (recursos que aún no terminaban de crearse antes de intentar eliminarse) fue un resultado valioso que en un ambiente de producción se traduciría en la necesidad de manejo de reintentos o colas.